Prueba técnica — React Developer Senior

Stack: React 18, TypeScript estricto. Puedes usar librerías públicas (React Router 6/7, TanStack Query 5, Zod, Vitest, Testing Library, MSW). No necesitas ningún framework ni design system concreto: HTML semántico basta.

Contexto

Trabajamos en una aplicación interna de **tickets de seguridad**. Un ticket tiene esta forma:

```ts
type TicketState = "REPORTED" | "IN_PROGRESS" | "PENDING_SECURITY" | "DONE";
type Severity = "CRITICAL" | "HIGH" | "MEDIUM" | "LOW";

interface Ticket {
  id: string;
  key: string;             // "SEC-1024"
  asset: string;           // "payments-api" o "None" si es desconocido
  assignedTeam: string | null;
  assignee: string;        // email
  severity: Severity;
  state: TicketState;
  targetDate: string;      // date-time RFC 3339, p. ej. "2026-11-30T23:59:59Z"
  dueDate: string | null;  // date-time RFC 3339 o null
  lastModified: string;    // date-time RFC 3339
}
```

Lo acordado con el equipo de backend es lo siguiente. **Todavía no existe un contrato formal**: la Pregunta 2 empieza por definirlo.

```
GET /api/v1/tickets?
offset=0&limit=20&q=log4j&state=IN_PROGRESS&severity=HIGH&sort=TARGET_DATE&direction=ASC
→ 200 { "data": Ticket[], "pagination": { "offset": 0, "limit": 20, "totalElements": 134 } }
→ 422 si algún parámetro es inválido (offset < 0, limit fuera de 1..100, enum desconocido)

PATCH /api/v1/tickets/{id}   body: { "dueDate": "2026-11-15T10:00:00Z" }
→ 200 Ticket actualizado. Si el ticket estaba en REPORTED pasa automáticamente a IN_PROGRESS.
→ 403 si el usuario no tiene el permiso "tickets.update"
→ 409 si el ticket está en DONE
```

Pregunta 1 — Revisión de código

Un compañero ha abierto una PR con esta página. Revísala como lo harías en una PR real:

1. Enumera los problemas que veas, **ordenados por gravedad**, explicando el impacto de cada uno para el usuario o para el mantenimiento.
2. Reescribe el componente (o las partes que consideres) con la solución que propondrías.

```
export function TicketsPage() {
  const { teamId } = useParams();
  const [searchParams, setSearchParams] = useSearchParams();
  const [tickets, setTickets] = useState<any[]>([]);
  const [total, setTotal] = useState(0);
  const [loading, setLoading] = useState(false);
  const [search, setSearch] = useState(searchParams.get("q") ?? "");

  const offset = parseInt(searchParams.get("offset")!);
  const limit = 20;
  const state = searchParams.get("state") || "ALL";

  useEffect(() => {
    setLoading(true);
    fetch(`/api/v1/teams/${teamId!}/tickets?
offset=${offset}&limit=${limit}&state=${state}&q=${search}`)
      .then((r) => r.json())
      .then((body) => {
        setTickets(body.data);
        setTotal(body.pagination.totalElements);
        setLoading(false);
      });
  }, [offset, state, search]);

  const changeState = (value: string) => {
    searchParams.set("state", value);
    setSearchParams(searchParams);
  };

  const expired = tickets.filter((t) => new Date(t.targetDate) < new Date());

  return (
    <div>
      <input value={search} onChange={(e) => setSearch(e.target.value)} />
      <select value={state} onChange={(e) => changeState(e.target.value)}>
        <option value="ALL">Todos</option>
        <option value="REPORTED">Reportado</option>
        <option value="IN_PROGRESS">En curso</option>
        <option value="DONE">Hecho</option>
      </select>
      {loading && <Spinner />}
      <p>{expired.length} tickets caducados</p>
      <table>
```

```
        <tbody>
          {tickets.map((t, i) => (
            <TicketRow key={i} ticket={t} onChange={() => setTickets([...tickets])} />
          ))}
        </tbody>
      </table>
      <Pagination
        total={total}
        offset={offset}
        limit={limit}
        onChange={(o: number) => {
          searchParams.set("offset", String(o));
          setSearchParams(searchParams);
        }}
      />
    </div>
  );
}
```

Pregunta 2 — Contrato OpenAPI y hook de filtros sincronizados con la URL

Parte A — Define el contrato

Antes de consumir la API, formalízala. Escribe un único fichero `openapi-rest.yml` (OpenAPI **3.0.x**, no 3.1) que describa las tres operaciones del contexto: `GET /api/v1/tickets`, `PATCH /api/v1/tickets/{id}` y `GET /api/v1/me`.

Requisitos del contrato:

1. Cada operación tiene `operationId` único y descriptivo, `tags`, `summary` y `description`.
2. Todo esquema, propiedad y parámetro lleva `description` y un `example` realista que valide contra su esquema.
3. Toda validación tiene límites: los `string` con `maxLength` o `pattern`, los `array` con `maxItems`, los números con `minimum` y `maximum`. Los parámetros opcionales tienen `default`.
4. Los valores de los `enum` van en MAYÚSCULAS, y `default` y `example` pertenecen al `enum`.
5. Nombres: rutas en minúsculas y `kebab-case`; parámetros y propiedades en `camelCase`; esquemas en `PascalCase` y sin sufijo `DTO`.
6. El listado devuelve `{ data, pagination }` con un esquema propio para la página y otro para `pagination`.
7. Todas las fechas (`targetDate`, `dueDate`, `lastModified`) con `format: date-time` (RFC 3339). Lo que puede ser `null` está declarado como tal.
8. Errores: declara solo los códigos que tienen sentido en cada operación, reutilizando respuestas definidas en `components/responses` con un esquema de error común (Problem Details, RFC 9457).
9. Seguridad: esquema `bearer` global y servidores solo HTTPS.
10. Todo vive en un solo fichero: los esquemas, parámetros y respuestas reutilizables van en `components`.

Además, responde brevemente: ¿cómo evitarías que los tipos de TypeScript de la aplicación y este contrato se desincronicen con el tiempo?

Parte B — Hook de filtros

Queremos que el listado sea **compartible por URL** y que funcionen los botones atrás/adelante del navegador. Los parámetros y sus valores válidos son los de tu contrato. Implementa un hook:

```ts
function useTicketListParams(): {
  params: TicketListParams;                          // valores ya validados y con defaults
  setFilters: (patch: Partial<Omit<TicketListParams, "offset" | "limit">>) => void;
  setPage: (offset: number) => void;
  reset: () => void;
};
```

Requisitos:

1. `TicketListParams` incluye `offset`, `limit`, `q`, `state`, `severity`, `sort` y `direction`. Defaults: `offset=0`, `limit=20`, `state=ALL`, `severity=ALL`, `sort=LAST_MODIFIED`, `direction=DESC`, `q` vacío.
2. La URL es la **única fuente de verdad**. Una URL manipulada (`?offset=-5&limit=9999&state=FOO`) nunca debe romper la página ni llegar así al backend: se normaliza a valores válidos.
3. Cambiar cualquier filtro o el orden **reinicia la paginación** (`offset=0`). Cambiar de página no toca los filtros.
4. Los parámetros con valor por defecto **no se escriben** en la URL (URLs limpias).
5. La búsqueda por texto `q` no debe lanzar una petición por cada pulsación (explica dónde pondrías el debounce y por qué).
6. Escribe los tests que consideres imprescindibles para este hook (Vitest + Testing Library, `renderHook` con un `MemoryRouter`).

Pregunta 3 — Mutación con actualización optimista y permisos

En cada fila hay una acción **"Fijar fecha compromiso"** que hace `PATCH /api/v1/tickets/{id}` con `dueDate`. Con TanStack Query (o la herramienta que prefieras, justificándola):

1. Implementa un hook `useSetDueDate()` con **actualización optimista**: la fila debe mostrar la nueva fecha y, si estaba en `REPORTED`, el estado `IN_PROGRESS` inmediatamente. Si el servidor falla, se revierte y se informa al usuario.
2. Ten en cuenta que puede haber **varias consultas del listado en caché** (distintos filtros, páginas). ¿Qué haces con ellas? ¿Qué pasa si la fila deja de cumplir el filtro activo (por ejemplo, el filtro es `state=REPORTED`)?
3. La acción solo debe mostrarse a usuarios con permiso `tickets.update` (el permiso llega en `GET /api/v1/me` → `{ permissions: string[] }`). Implementa cómo lo resolverías en la UI y responde: **¿es suficiente con ocultar el botón?**
4. Describe (o escribe) cómo probarías los casos 200, 403 y 409 sin backend real.

Entrega

- Código TypeScript compilable (puede ser en un único fichero por pregunta o un repo pequeño).
- `openapi-rest.yml` con el contrato de la Pregunta 2.
- Explicaciones breves en texto donde se piden.